

CSCE 3301 – Computer Architecture

Spring 2026

Project 2: femTomas

Tomasulo Algorithm Simulation

Project Overview: The goal of this project is to implement an architectural simulator capable of assessing the performance of a simplified out-of-order 16-bit RISC processor that uses **Tomasulo's algorithm without speculation**. This document details the instruction set to be supported, the inputs to the simulator, and the expected outputs

Implementation language: Any general-purpose programming language. The resulting application can either be a console application or a graphical user interface (GUI) application (desktop, web-based, or mobile app) as a bonus feature.

Instruction set architecture (ISA): The simulator assumes a simplified RISC ISA inspired by the ISA of the Ridiculously Simple Computer (RISC-16) proposed by Bruce Jacob. As implied by its name, the word size of this computer is 16-bit. The processor has 8 general-purpose registers R0 to R7 (16-bit each). The register R0 always contains the value 0 and cannot be changed. Memory is **word addressable** and uses a 16-bit address (as such the memory capacity is 128 KB). The instruction format itself is not very important to the simulation and therefore is not described here. However, the simulator should support the following set of instructions (16-bit each):

1. Load/store

- Load: Loads a word from memory into `rA`. Memory address is formed by adding `offset` to contents of `rB`, where `offset` is a 5-bit signed constant (ranging from -16 to 15).
 - `LOAD rA, offset(rB)`
- Store: Stores value from `rA` into memory. Memory address is computed as in the case of the load word instruction
 - `STORE rA, offset(rB)`

2. Conditional branch

- Branch if equal: branches to the address `PC+1+offset` if `rA=rB`. Note that the PC is incremented by one when the branch is not taken.
 - `BEQ rA, rB, offset`

3. Call/Return

- Call: Stores the value of `PC+1` in `R1` and branches (unconditionally) to the address specified by the `label`. The label is encoded as a 7-bit signed constant.
 - `CALL label`
- Return: branches (unconditionally) to the address stored in `R1`
 - `RET`

4. Arithmetic and Logic

- Add: Adds the value of `rB` and `rC` storing the sum in `rA`
 - `ADD rA, rB, rC`
- Sub: Subtracts the value of `rC` from `rB` storing the difference in `rA`
 - `SUB rA, rB, rC`
- And: Performs a bitwise and operation between the values of `rB` and `rC` storing the result in `regA`
 - `AND rA, rB, rC`
- Mul: Multiplies the value of `rB` by `rC` storing the least significant 16-bit of the 32-bit result in `rA`.
 - `MUL rA, rB, rC`

Simulator inputs:

1. The assembly program to be simulated. The user should also specify its starting address. Instructions can be entered one by one or even selected from a list.
2. Data required by the program to be initially loaded in the memory. For each data item both its value and its memory address should be specified. All values are assumed to be 16-bit values.

Simulation: The simulator should simulate the program's execution on a single-issue processor (multiple-issue simulation is a bonus feature). It should follow the non-speculative version of Tomasulo's algorithm. The simulator should only take the **3 backend stages** into account: issue (1 cycle), execute (N cycles depending on the functional unit involved as detailed in the table below), write (1 cycle except for stores as outlined below)

You can assume that all instructions have already been fetched and decoded and are waiting to be issued.

The following is assumed about the functional units available:

	Number of reservation stations available	Number of cycles needed
LOAD unit	2	2 (compute address) + 6 (read from memory)
STORE unit	2	2 (compute address) + 6 (writing to memory)
BEQ unit	2	1 (compute target and compare operands)
CALL/RET unit	1	1 (compute target and return address)
ADD/SUB unit	4	2
AND unit	2	1
MUL unit	1	8

Do not ignore load-store hazards. For conditional branches, the processor uses an always not taken predictor. Recall that in the non-speculative version of Tomasulo, instructions can be issued (but not executed nor written) based on a prediction.

While simulating the execution, the program should record the number of instructions completed, the number of conditional branches encountered, the number of cycles spanned, and the number of branch mispredictions.

Simulator output: At the end, the simulator should display the following performance metrics:

1. A table listing the clock cycle time at which each instruction was: issued, started execution, finished execution, and written
2. The total execution time expressed as the number of cycles spanned
3. The IPC
4. The branch misprediction percentage

Simplifying assumptions:

1. Fetching and decoding take 0 cycles and the instruction queue is already filled with all the instructions to be simulated.
2. No floating-point instructions, registers, or functional units
3. No input/output instructions are supported
4. No interrupts or exceptions are to be handled
5. For each program being executed, assume that the program and its data are fully loaded in the main memory
6. There is a one-to-one mapping between reservation stations and functional units. i.e., Each reservation station has a functional unit dedicated to it
7. No cache or virtual memory (see bonus 7)

Bonus features:

1. Building the application as a GUI application (desktop, web-based, or mobile app)
2. Building the application as an educational GUI application. In this case the user should not only be allowed to simulate entire programs at once and get performance metrics but also to step through the program cycle-by-cycle while monitoring the internals of the processor (the status of all reservation stations, the register status table, the register file, and all memory locations holding relevant data values). This feature is also helpful for debugging purposes and will be **counted as two features** as far as grading is concerned (since it includes and extends the previous bonus feature).

3. Support a variable hardware organization. If implemented the user should specify the number of reservation stations for each class of instructions and the number of cycles needed by each functional unit type.
4. Support a multiple-issue hardware organization. If implemented the user should specify the pipeline width.
5. Implementing and integrating a parser to allow the user to provide the input program using proper assembly language including labels for jump targets and function calls. The assembly program can be entered directly in the application or in a text file read by the application.
6. Implement **two** dynamic branch prediction algorithms (1-bit and 2-bit) and compare their effects on the performance.
7. Consider the effect of the **data** cache hierarchy. This bonus will also be **counted as two features** as far as grading is concerned. If implemented, the user of the simulator should enter the number of cache levels he wants to simulate. For each cache level, the user should specify: 1) the full cache geometry (S, L, m), 2) writing policies (both in case of hit and miss), and 3) the number of cycles required to access data. The user should also specify main memory access time (in cycles). In this case, the simulator should assume that the cache is initially cold (nothing is cached at the beginning of a program execution) and simulate the memory hierarchy behavior recording the number of cache accesses and misses in each cache level so that it can produce the following extra outputs:
 - a. The hit ratio of each cache level
 - b. The global AMAT of the memory hierarchy (in cycles)

Important Plagiarism notice: You have to write your own code from scratch. Submissions based on others code will receive a grade of **zero** in the entire project (even if you understand every code line and even if the code is heavily re-factored/modified). Examples of such sources include (but is not limited to) code coming from the following sources: other teams, previous course offerings, open-source software, tutors, Internet, etc.

Important AI usage notice: You **can** use AI-generated code provided the project is not in its entirety AI-generated and provided you fully understand the generated code and have properly tested it. Also, you have to document the tools and the exact process you followed in your report specifying which parts were AI generated and how they were generated (by providing used prompts).

Deliverables and Deadline: There are no milestones for this project. You are required to submit the full project at once on **Tuesday May 12, 2026 (11:59 pm)**. **Demos will be scheduled on Wednesday May 13.**

Grading Criteria

- As already announced, both projects (project1 and project2) collectively are worth 40% of the course marks. This project will count for 15% of the course marks while project 1 will count for 25%
- Bonuses: Each bonus feature will count for 5% with a maximum of 2 bonuses worth 10%. Please do not be tempted to implement more than 2 bonus features, as this will cost you too much time.
- Deductions:
 - -5% for not following the required submission directory structure.
 - -5% for late submissions (1 day only).
 - -100% for plagiarized submissions.

General Guidelines

- Preferably work in the same teams as for Project1 or inform me by email of any changes
- Every team member must log all her/his activities in a journal (a text file). For example:
May 5, 1:00PM: implemented a function to detect instructions dependencies.
- Final project must be submitted by the group leader as a single **zip** or **rar** file including the following:
 - A readme.txt file that contains student names and IDs as well as release notes (issues, assumptions, what works, what does not work, etc.)
 - Journals folder: has the group journals
 - Source Code folder: Simulator code in the general-purpose language of your choice
 - Test folder: The assembly programs you used to test your program (and associated data if any). You should at least provide 3 programs. The programs must cover all instructions supported and one of them at least must have a loop.
 - A report file including:
 - A brief description of your implementation including any bonus features included.

- A detailed description of any AI usage.
- A user guide including a full simulation example step-by-step with snapshots.
- The results obtained from the simulation of each assembly program provided.
- A brief discussion of the obtained results.
- Optionally, you can include a section about your experience working on this project. This section will NOT affect your grade in any way.